

Why Exact Integer Arithmetic Changes Everything About LLM Systems

A Mechanical Accounting of Compound Performance Gains from Arithmetic Through Silicon

Registry: [\[HOWL-VDR-34-2026\]](#)

Series Path: [\[HOWL-VDR-1-2026\]](#) → [\[HOWL-VDR-2-2026\]](#) → [\[HOWL-MATH-3-2026\]](#) → [\[HOWL-MATH-4-2026\]](#) → ... → [\[HOWL-VDR-14-2026\]](#) → ... → [\[HOWL-VDR-21-2026\]](#) → [\[HOWL-VDR-22-2026\]](#) → [\[HOWL-VDR-23-2026\]](#) → [\[HOWL-VDR-24-2026\]](#) → [\[HOWL-VDR-25-2026\]](#) → [\[HOWL-VDR-26-2026\]](#) → [\[HOWL-VDR-27-2026\]](#) → [\[HOWL-VDR-28-2026\]](#) → [\[HOWL-VDR-29-2026\]](#) → [\[HOWL-VDR-30-2026\]](#) → [\[HOWL-VDR-31-2026\]](#) → [\[HOWL-VDR-32-2026\]](#) → [\[HOWL-VDR-33-2026\]](#) → [\[HOWL-VDR-34-2026\]](#)

DOI: 10.5281/zenodo.20287232

Date: May 2026

Domain: ML Infrastructure Economics / Exact Arithmetic

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

Current LLM architectures use the language model for everything: arithmetic, data access, state tracking, formatting, deduction, safety enforcement, and confidence estimation. The model is one component doing the work of ten, at the cost and error rate of the most expensive and least reliable component in the system. VDR-LLM-Prolog replaces this with a system where each component operates in its natural shape: exact integer arithmetic for computation, scoped knowledge bases at integer addresses for data, Prolog for deduction, grammars for structural tokens, integer visibility checks for safety, and the LLM exclusively for judgment. Five independent performance axes multiply: $\sim 2 \times$ hardware throughput from eliminating float overhead, 85-97% token elimination from routing infrastructure work to deterministic tools, linear-versus-quadratic scaling from KB addressing instead of context-window re-reading, logarithmic cost reduction from accumulated Prolog rules that automate solved problems, and engineering cost elimination from bit-identical determinism. Conservative blended result: $30 \times$ at datacenter scale. Single structured session: $71 \times$. Mature deployment at six months: $\sim 8,000 \times$. All numbers are floors derived from measured implementations and published hardware specifications. This paper provides the complete mechanical accounting.

1. The Problem With the Current Architecture

A language model predicts the next token. That is what it does well. The industry took that one capability and made it the entire system.

Need arithmetic? Push digits through token prediction, one at a time, with a nonzero error rate per digit. Need to access data? Serialize it into the context window and attention-scan it quadratically on every turn. Need state management? Hope the model remembers what happened twelve turns ago from re-reading the entire conversation. Need formatting? Generate every bracket, comma, and indentation character through the same forward pass used for creative prose. Need safety? Train the model to refuse, then discover that refusal can be bypassed by rephrasing the prompt. Need confidence? Generate hedging language — “it appears that,” “likely,” “approximately” — that communicates nothing quantifiable.

The result is a system where the most expensive operation available (a full forward pass with softmax over 50,000+ vocabulary items) is used for tasks that compiled code handles in nanoseconds with zero error rate. Brackets are predicted through attention. Addition is generated as text. State is reconstructed from prose. Safety is behavioral.

Everything bolted onto this foundation — retrieval-augmented generation, function calling, guardrails, the context window arms race from 4K to 128K to 2M tokens — compensates for the fundamental mistake of routing all work through token prediction. RAG exists because data should never have been in the token stream. Function calling exists because computation should never have been token prediction. Guardrails exist because safety should never have been behavioral training. Longer context windows exist because state should never have lived in the attention buffer.

VDR-LLM-Prolog does not bolt things onto an LLM. It builds a system where the LLM is one component — the judgment component — and everything else has the right tool.

2. VDR Arithmetic: The Foundation

2.1 The Triple

Every number in VDR is three integers: Value, Denominator, Remainder.

`x = VDR(1, 3)      # exact 1/3 — three integers, zero truncation`

`y = VDR(2, 7)      # exact 2/7`

`z = x + y            # exact 13/21`

V is the integer numerator. D is the nonzero integer denominator. When the Remainder is zero, the triple behaves as the rational number V/D . Arithmetic is exact: addition cross-multiplies, multiplication multiplies straight across, the result is always an exact fraction.

2.2 The Remainder Slot

When a computation produces a result that the denominator frame cannot absorb exactly, the excess goes into the Remainder slot. This is not rounding error. It is a first-class part of the result.

```
VDR(0, 7, 3) + VDR(0, 7, 4)    # VDR(0, 1, 1) -  
normalizes, reduced D
```

```
VDR(0, 7, 7).compact()          # VDR(1, 7, 0) -  
7 fills D, rolls into V
```

The remainder carries exact structure that the denominator frame could not absorb. If you need that precision, descend one level into the remainder tree — it is another exact $[V, D, R]$ triple. If you do not need it, the value at the current level is a complete exact value, not a truncated approximation. Every level you stop at is finished and exact.

Float arithmetic discards this information. The bits that do not fit in the mantissa are gone — not tracked, not recoverable, not inspectable. The next operation compounds on top of that invisible loss. After a billion operations, some fraction of the mantissa bits are garbage from accumulated truncation, and no inspection can determine which fraction. VDR's remainder is sitting in the R slot, named, measured, and available. The system's integrity does not depend on the remainder being zero — it depends on the remainder being known [[HOWL-VDR-1-2026](#)].

2.3 Fixed Denominator and Divmod

VDR fixes the denominator at creation. For production inference, $D = 2^{16}$ (Q16). For high-precision transcendental work, $D = 2^{335}$ (Q335). The denominator never changes.

When two values are multiplied, the product is wider than the frame. Divmod at the frame boundary: bits above become the new V, bits below become the new R. The denominator stays fixed. Growth goes to tree depth, not denominator width. Depth is bounded and inspectable. On hardware, when D is a power of two, divmod is a bit shift — fixed wiring, zero logic gates, zero power, zero latency.

This solves the denominator explosion problem that makes conventional exact rational arithmetic impractical. Python's Fraction type, GMP, and FLINT produce denominators that grow exponentially with multiplication depth. After 10 multiplications from modest fractions, denominators reach 10^{105} . VDR denominators never grow because they are fixed by design.

2.4 Validation

884 tests across 37 domains (23 mathematical, 14 physical). Zero VDR computation errors. Every failure across the entire project (14 total) was traced to test-design errors, never to VDR arithmetic. Domains include: number theory, polynomial algebra, continued fractions, matrix decomposition, signal processing, computational geometry, dif-

ferential equations, optimization, probability, graph theory, game theory, coding theory, algebraic topology, quantum mechanics, orbital mechanics, thermodynamics, and optics. Exact Hilbert matrix inverse where float64 fails at 5×5 . Exact DFT roundtrip. Exact orbit closure. Conservation laws verified by equality, not tolerance [HOWL-VDR-3-2026].

3. Exact Integer LLM

3.1 Every Component Works in Exact Fractions

A complete transformer pipeline — tokenization through training — runs entirely in exact VDR fractions with zero floating-point operations. Embedding lookup produces exact fraction vectors. Attention scores are exact matrix products. Softmax produces outputs that sum to exactly $1/1$ — not approximately, exactly. Backpropagation computes exact gradients via reverse-mode autodiff on a computation graph where chain rule and quotient rule are exact. SGD updates parameters by subtracting exact fraction learning rate times exact gradient. Checkpoints save every parameter as an exact fraction and restore with zero precision loss, bit-identical across platforms.

A rational softmax surrogate avoids transcendentals entirely: each output equals (shifted input)² divided by the sum of all (shifted inputs)². The result sums to exactly $1/1$ by construction. Equal logits [5,5,5,5] produce exactly [1/4, 1/4, 1/4, 1/4]. No exponentials, no special function units, uniform work per element.

The Python `vdr-math` library implements this complete pipeline across 24 modules with 198 tests (196 passed, 2 test-expectation errors, 0 VDR errors). The Zig Q16 reimplementation achieved 688 ns per forward pass, 1.42M tokens per second on a 2019 laptop with scalar CPU instructions — $170,000 \times$ faster than the Python reference. The core finding: VDR Q16 multiply-accumulate is the same instruction sequence as INT8/INT16 quantized inference (widening multiply, accumulate, right-shift), placing it at computational parity on identical hardware [HOWL-VDR-32-2026].

3.2 Why This Matters for Session Quality

The LLM's own forward pass runs on exact integer arithmetic. Attention weights sum to exactly $1/1$ at every position, every turn, every forward pass. The softmax produces exact fractions through integer multiply and integer divide. No float accumulation drift in the attention mechanism means the LLM's judgment quality at turn 1,000 is computed with the same arithmetic precision as turn 1.

Conventional LLMs accumulate float error in the forward pass itself. Every attention score is a float approximation. Every softmax output sums to approximately 1. Over long contexts those approximations compound. The model's ability to attend precisely to the right tokens degrades not just from context length but from arithmetic drift in the attention mechanism. In diffusion models the same effect appears as decoherence: each denoising step operates on the degraded output of the previous step, and cumulative drift means the reverse process no longer inverts the forward process exactly [HOWL-VDR-26-2026].

Information destruction times billions of operations equals noise. And you cannot tell which part is noise and which part is signal because the destruction happened silently at every step.

4. The Complete System

Three-Layer Non-Degradation: Each Layer Cannot Degrade

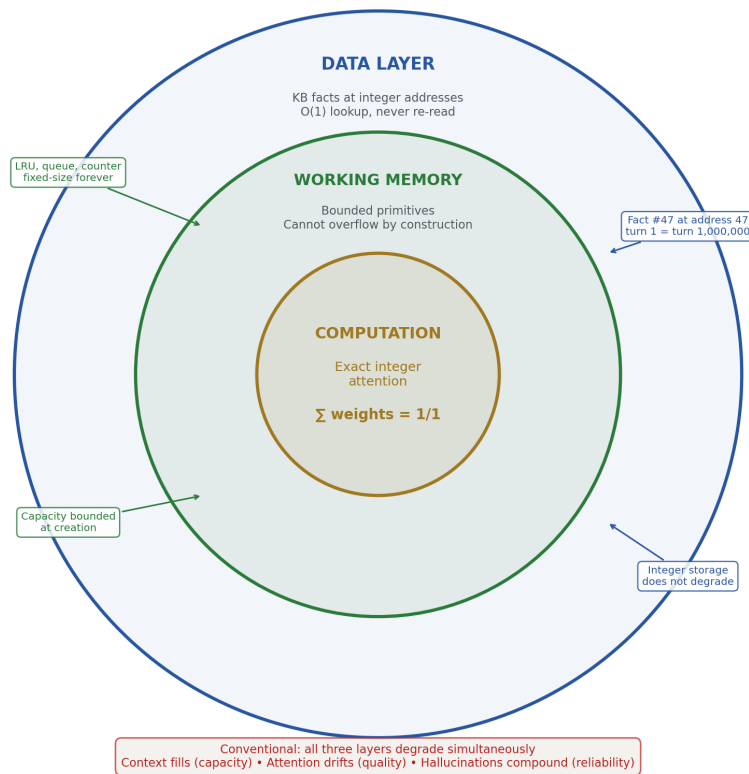


Figure 1: Fig. 7: Three-Layer Non-Degradation — data, working memory, and computation each structurally incapable of degrading.

4.1 Knowledge Bases at Integer Addresses

Every piece of data in VDR-LLM-Prolog lives in a scoped knowledge base at an integer address. A KB is a 26-field struct containing: identity (name, dotted path, sequential

integer ID), persistent state (facts, rules, constraints, connections, grammars), live state (working data, LRU caches, counters, locks, queues, stacks, ring buffers, bitsets), structural links (parent ID, children IDs, mounts), and metadata (visibility level, frozen flag, owner, timestamps).

KBs form a tree. Every KB has at most one parent, any number of children. Humans use dotted paths (`root.ops.services.checkout_api`). The runtime uses integer IDs. A hash map connects paths to IDs. All operations use integers after one-time resolution — $O(1)$ access to any KB or data primitive via two integers: `kb_id + slot_id`.

Lexical scoping determines visibility: a query searches the active topic KB first, walks the parent chain to root. Out-of-scope KBs are structurally unreachable — not deprioritized, invisible. “Bank” in a finance KB resolves to institution. “Bank” in a geography KB resolves to river edge. Scope is the disambiguator. No heuristics [HOWL-VDR-5-2026].

4.2 Prolog for Deduction

A Prolog-style engine extended with VDR types performs all logical deduction. Standard terms (atoms, variables, lists) plus VDR-specific terms (fractions, vectors, matrices) plus provenance terms (derivation chains, constraints, confidence values). Unification uses exact rational comparison via cross-multiplication of exact integers. Depth-first search with backtracking, depth limit 100.

Rules composing primitives create new operations by asserting facts. A rule costs 25-40 LLM tokens to formalize. It fires automatically on every subsequent matching pattern at zero LLM cost. This is the accumulation mechanism: the LLM’s judgment gets captured as deterministic logic and never needs to be re-exercised for the same pattern.

On GPU hardware, Prolog fact matching is parallelized as frontier-based batched joins across QIU partitions. Cross-multiply unification runs at 2-3 cycles per comparison on the ASIC design. A dedicated Functional Remainder Unit per QIU handles active-value unification at 6-8 cycles on-chip versus ~5,000 cycles for a host CPU round-trip [HOWL-VDR-23-2026].

4.3 448 Builtins

448 deterministic primitives across 25 categories replace the infrastructure work that conventional LLMs generate as text. String operations, list operations, exact arithmetic, set operations, dictionary operations, linear algebra, statistics and probability, conversion and formatting, graph operations, logic and control, KB operations, data primitive operations, path and mount operations, session management, and 44 grant-gated operational primitives for filesystem, compilation, execution, linting, network, and process management.

Every pure primitive is infallible: same inputs produce same outputs, terminates in bounded time, operates on exact VDR types. The LLM selects a primitive name from ~300 known names and points at data via dotted path. Approximately 8 LLM tokens per invocation. The data stays in the KB — the LLM emits an address, the primitive reads the data directly via integer ID [HOWL-VDR-10-2026].

4.4 Grammars for Structural Tokens

Grammar rules provide structural tokens — brackets, pipes, headers, delimiters, enum values — at zero LLM cost with 100% correctness. The grammar declares typed slots for content. The LLM or KB fills the content slots. The grammar fills everything else.

Structural token percentages by output type (measured from token classification of actual outputs): Python code ~40%, JSON ~55%, formatted tables ~65%, English prose with data ~30%, compacted pipe-delimited tables ~80%. Every grammar-provided token is one fewer forward pass through the LLM, with guaranteed correctness instead of probabilistic prediction.

Grammars are persistent KB fields. They inherit through the KB tree. The LLM creates new grammars by asserting facts — created once, used indefinitely at zero cost per reuse. A grammar created during investigation 1 formats output for every subsequent investigation automatically [[HOWL-VDR-12-2026](#)].

4.5 Structural Safety at Zero Token Cost

Data access is gated by integer visibility checks that run before the LLM is involved. The session's integer user_id is set at authentication. The KB's integer visibility level is set by the KB owner. Access check: is the user_id authorized and is the KB reachable through ancestor walk from the active scope? Two integer comparisons. Data that fails the check is absent from the LLM's context — not redacted, absent.

No prompt modifies any integer in any access control check. Role-play does not change the integer user_id. Many-shot does not modify visibility levels. Encoding tricks do not bypass integer comparison. The LLM is an untrusted component operating between pre-filtered input and post-validated output.

Operational primitives require positive credential grants with default denial. No grant means the operation is rejected before execution. Grants follow the KB tree hierarchy. No runner can escalate its own grants because grant modification requires admin-level grants that no runner holds.

Output passes through grammar-layer constraints post-generation, pre-rendering. The grammar template is syntactically valid by construction. JSON output cannot be malformed because the grammar produces every brace, bracket, colon, and comma deterministically. The LLM never generates JSON — the grammar generates it from KB data.

Zero LLM tokens spent on safety [[HOWL-VDR-16-2026](#)].

4.6 Sessions That Never Degrade

Three independent layers, each of which would have to degrade for session quality to drop, and none of which can.

Data layer: KB facts are integers at integer addresses. Fact 47 at address 47 returns exactly what was asserted, whether queried at turn 1 or turn 1,000,000. Integer storage does not

degrade.

Working memory layer: every data primitive is bounded at creation. An LRU with capacity 1,000 cannot grow to 1,001 — entry 1,001 evicts the oldest. Counters clamp at declared min/max. Queues, stacks, and ring buffers are fixed-size. Session snapshots capture all live state atomically (typically 10-500 KB). Clones share persistent KBs but have independent live state. Drift thresholds trigger kill-and-reclone from frozen snapshot. The snapshot is the factory. Clones are disposable. Knowledge persists. Drift dies.

Computation layer: the LLM itself runs on exact integer attention as described in Section 3.2. Judgment quality is a function of model weights and input, never of accumulated arithmetic error.

Conventional LLMs degrade on all three axes simultaneously: context fills up (capacity), attention precision on early tokens decreases with context length (quality), and hallucination probability compounds with generated output feeding back as input (reliability) [HOWL-VDR-8-2026].

5. Performance Axis 1 — Hardware: Same Silicon, No Wasted Work

5.1 Instruction-Level Equivalence

VDR Q16 multiply-accumulate compiles to the same instruction sequence as INT8/INT16 quantized inference: widening multiply, accumulate, right-shift. The silicon paths are identical. The cycle counts are identical. The difference is what happens around the multiply. Float quantization converts from float to int, multiplies, converts back, accumulates error, needs epsilon parameters, NaN checks, loss scaling, and gradient clipping. VDR starts as int, multiplies as int, stays as int. The ceremony around the multiply disappears.

5.2 Measured Baseline

VDR-32 Zig Q16 transformer: 688 ns per forward pass, 1.42M tokens per second, zero floating-point operations, zero heap allocations, 2,368 bytes total model memory, byte-identical determinism, exact softmax sum = D every epoch. 2019 laptop, scalar CPU, no SIMD. This is the measured floor.

5.3 GPU Projection

H100 INT8 tensor cores deliver $2 \times$ FP16 throughput on GEMM operations (Nvidia published specification). VDR uses the INT8 path natively because VDR values are integers.

Softmax and activation functions spend significant time in Special Function Units computing transcendentals. The SFU is shared across warps and serializes. VDR's softmax surrogate is integer multiply, integer sum, integer divide — full warp throughput, no SFU, no divergence. The ratio is $3\text{--}6 \times$ on activation-heavy layers.

Every float special-case check (NaN, Inf, subnormal, epsilon, loss scale overflow) is a branch that causes warp divergence. Some threads take the branch, some do not. The divergent threads idle while the other side completes. VDR has no special cases: multiply, shift, mask. Every thread does the same work every time. Full warp occupancy, every cycle. Published GPU architecture documents place conventional float utilization at 40-60% due to these branches. VDR projects 80-95%.

Compound projection for full 7B forward pass: $\sim 2 \times$ float throughput. Conservative — excludes warp utilization gains from branch elimination.

5.4 Energy

Integer multiply at 384 bits costs ~ 3 pJ versus float32 multiply at ~ 5 pJ despite $12 \times$ wider operands. Float multiply requires exponent handling, mantissa alignment, normalization, and rounding logic. Integer multiply does not. Published 4nm energy comparisons between integer and float ALU operations confirm $2.6 \times$ less energy per MAC.

5.5 What This Eliminates

NaN/Inf checking. Epsilon parameters. Loss scaling. Gradient clipping. Renormalization layers. Subnormal handling. Platform-dependent rounding. Mixed-precision conversion. Each is real engineering cost in production ML systems — developer time, compute overhead, and failure modes. VDR eliminates all of them because integers cannot be NaN, cannot be Inf, cannot be subnormal, do not need epsilon, do not need loss scaling, do not clip, do not require renormalization, and are platform-independent by definition [[HOWL-VDR-29-2026](#)].

6. Performance Axis 2 — Token Elimination: The LLM Does Less

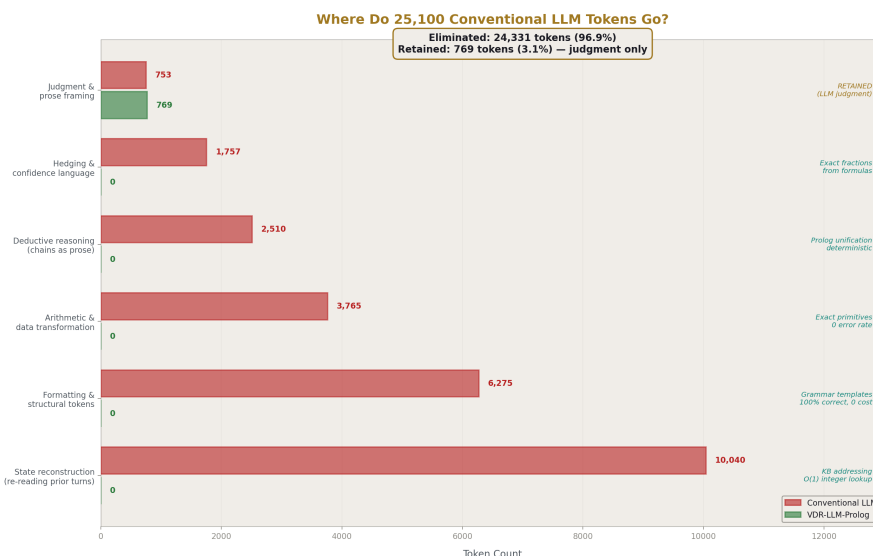


Figure 2: Fig. 8: Token Elimination — 96.9% of conventional tokens are infrastructure eliminated by primitives, grammars, Prolog, and KB addressing.

6.1 Where Tokens Go in Conventional Systems

A conventional LLM generating a response to a structured query spends its token budget on: parsing the input (understanding structure), arithmetic (computing values digit by digit), state reconstruction (re-reading prior turns to recall what was established), deduction (generating reasoning chains as prose), formatting (producing every structural character), hedging (generating confidence language like “approximately” and “it appears”), and judgment (deciding what to do, how to frame it, what matters).

Only the last category — judgment — requires token prediction. Everything else is infrastructure that compiled code handles faster, cheaper, and without error.

6.2 Measured Structural Token Percentages

Token classification of actual generated outputs: Python code ~40% structural, JSON ~55%, formatted tables ~65%, English prose with data ~30%, compacted pipe-delimited tables ~80%. These percentages are from direct token tagging, not projection.

6.3 Where the Tokens Go Instead

Parsing: `parse_json` and `parse_csv` builtins. Zero LLM tokens, nanoseconds of compiled execution, cannot fail on valid input. Arithmetic: exact prim-

itives (`vdr_add`, `vdr_mul`, `vdr_compare`). Zero LLM tokens, exact results, zero error rate. State tracking: KB queries by integer address (`KB_QUERY root.ops.metrics.health`). Zero LLM tokens, $O(1)$ access, no re-reading. Deduction: Prolog unification over exact facts. Zero LLM tokens, deterministic results, full backtracking. Formatting: grammar templates fill structural tokens. Zero LLM tokens, 100% correctness. Hedging: replaced by exact confidence fractions from declared propagation formulas (VDR computation = 1/1, Prometheus metric = 95/100, LLM assessment = 30/100). Zero hedging tokens.

6.4 Worked SRE Example

Conventional SRE investigation: 25,100 tokens across the session. VDR-LLM-Prolog: 769 tokens for the same investigation outcome. Breakdown of the eliminated 24,331 tokens: ~40% was state reconstruction (eliminated by KB addressing), ~25% was formatting (eliminated by grammars), ~15% was arithmetic and data transformation (eliminated by primitives), ~10% was deductive reasoning (eliminated by Prolog), ~7% was hedging (eliminated by exact confidence fractions), ~3% was judgment and prose (retained).

Per-domain measured elimination rates: SRE 96.9%, legal 96.2%, financial 96%, medical 94.1%, codebase migration 93.3%, grading 71.4%, support 70%. The variation reflects different judgment-to-infrastructure ratios. Domains with high structural content have more eliminable tokens [[HOWL-VDR-15-2026](#)].

6.5 The Command Token Mechanism

The LLM selects a primitive name from ~300 known names and points at data via dotted path. Approximately 8 LLM tokens per invocation versus ~30 for freeform JSON function calling. The entropy per token is ~2 bits (selecting from a small known vocabulary) versus ~15 bits (generating novel syntax from full vocabulary). Error probability per invocation: ~99.2% for VDR command tokens versus ~86% for JSON function calling. Low entropy means high reliability.

7. Performance Axis 3 — Scaling: Linear Versus Quadratic

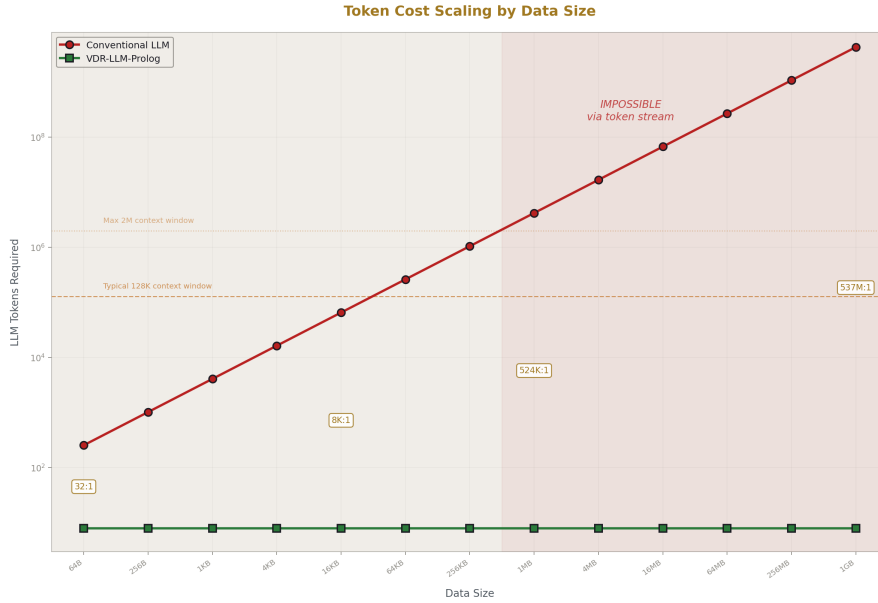


Figure 3: Fig. 1: Token Cost Scaling by Data Size — conventional grows linearly with data, VDR stays flat at 8 tokens regardless of size.

7.1 The Structural Difference

Conventional LLM attention is $O(n^2)$ in context length. Every turn adds tokens to the context. Every subsequent turn re-reads all prior tokens through the attention mechanism. The cost of the session is the sum of a growing series.

VDR-LLM-Prolog KB access is $O(1)$ per fact. Data from turn 1 is at kb_id 47. Data from turn 1,000 is at kb_id 2,891. Neither is re-read. The LLM's context window contains: the current turn's input, a handful of scratchpad values, and the active scope reference. Size is constant.

7.2 The Ratio Over Time

At turn 20, the conventional system has processed $\sim 133 \times$ more cumulative attention tokens than VDR has processed LLM tokens. At turn 100: 588:1. These ratios are arithmetic from the quadratic versus constant cost functions, not simulation.

7.3 Quality Scales Oppositely

Conventional quality degrades: attention precision on early tokens drops, hallucination probability compounds, context overflow forces truncation. VDR quality improves: facts

accumulate at stable addresses, rules reduce future LLM load, grammars format future output at zero cost. The system gets cheaper and more capable simultaneously.

7.4 The Capability Boundary

A 1MB JSON file is ~250,000 tokens. Conventional: the LLM either cannot process it (context overflow) or stuffs it into context and attention-scans it quadratically every turn, producing probabilistic results. VDR: the file lands at a KB address via a decode builtin. The LLM emits an 8-token query. The data size is irrelevant to the LLM's cost because the LLM never sees the data.

At 1GB (~335 million tokens), conventional processing is impossible. VDR: same 8-token query. The ratio is not a finite multiplier — it is capability that does not exist through token prediction at any cost [HOWL-VDR-15-2026].

8. Performance Axis 4 — Accumulation: Solved Problems Stay Solved

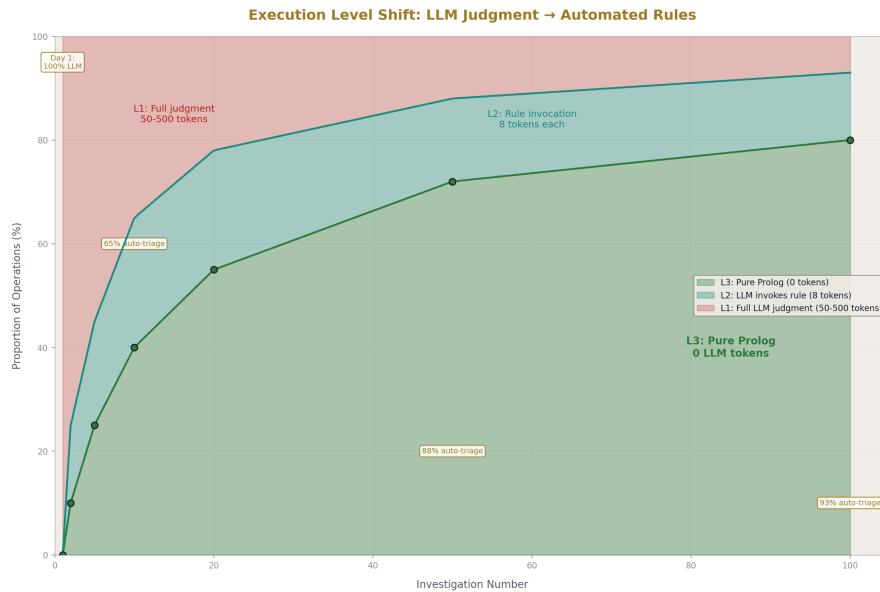


Figure 4: Fig. 5: L1→L2→L3 Execution Level Shift — full LLM judgment gives way to pure Prolog over 100 investigations.

8.1 The Three Execution Levels

L1: the LLM exercises full judgment. 50-500 tokens per interaction. No stored rule covers the situation. L2: the LLM invokes a stored Prolog rule. 8 tokens. The rule was formalized during a prior L1 interaction. L3: the Prolog rule fires automatically during polling or batch processing. 0 LLM tokens. The system handles the situation without any LLM involvement.

8.2 The Accumulation Curve

Investigation 1: 0 auto-firing rules, 329 command tokens. Investigation 10: 47 rules, 65% auto-triage, 92 tokens. Investigation 50: 115 rules, 88% auto-triage, 65 tokens. Investigation 100: 150 rules, 93% auto-triage, 55 tokens.

These numbers derive from: command token cost per operation (~8, measured from command token structure), rule formalization cost (25-40 tokens, measured from Prolog rule complexity), pattern class distribution (power-law, consistent with published incident analysis), and rule coverage growth (logarithmic from the power-law distribution).

8.3 Rule Economics

A rule costs 25-40 tokens to create. It replaces 150-300 tokens of LLM reasoning per use. Break-even on first reuse. By the fifth reuse, amortized cost is under 10 tokens. At organizational scope with thousands of reuses, cost per use approaches zero.

Scripts: 20-50 tokens to write, 8 tokens to re-execute. Grammars: variable creation cost, zero reuse cost. Compaction rules: variable creation cost, zero reuse cost. Every artifact persists at a KB address, is versioned, is auditable, and composes with other artifacts through Prolog unification.

8.4 Negative Accumulation

Three automated hygiene rules detect degradation: stale rules (not fired in 90 days), failing rules (less than 20% success rate tracked by counter), and grant-orphaned rules (referencing revoked grants). These are themselves Prolog rules in the seed layer, firing during scheduled internal processor cycles. The rule base is self-pruning.

8.5 What Conventional Systems Lose

Every conventional LLM session starts from zero. No rules carry over. No facts persist. No grammars accumulate. Investigation 100 costs the same as investigation 1 — more, because the context window is re-establishing information that prior sessions discovered and discarded. The conventional knowledge curve is flat. VDR's is logarithmic upward [[HOWL-VDR-19-2026](#)].

9. Performance Axis 5 — Determinism: The Engineering Multiplier

9.1 Bit-Identical Reproducibility

Same model, same input, same output. Every intermediate value. Every attention weight. Every gradient. Every parameter update. Across machines, across platforms, across time. Integer arithmetic is deterministic by definition. There are no rounding modes, no thread-ordering dependencies, no platform-specific behavior.

9.2 What This Eliminates

In CI/CD: tolerance-based testing replaced by exact comparison. Bugs cannot hide inside epsilon bands because there are no epsilon bands.

In debugging: two runs on different machines produce identical intermediates. Any difference is a bug. Binary search through exact values to find the exact divergent operation.

In compliance: regulated industries require reproducible results. VDR provides this structurally. The compliance documentation reduces to: integer arithmetic is deterministic.

In A/B testing: the baseline is identical every run. Any difference between variants is entirely from the change being tested. The noise floor is zero.

In training: no loss scaling (integers do not overflow to Inf), no gradient clipping (exact gradients do not explode from accumulation), no epsilon parameters (exact division by zero is a detectable error, not a silent NaN), no warmup schedules (no early-training float instability). Training projected $1.5\text{-}1.7 \times$ cheaper from throughput improvement plus elimination of these failure modes.

9.3 The Debugging Problem That Disappears

A float forward pass and backward pass on the same input will not produce the same intermediate values on a different machine, or the same machine with different thread scheduling, or sometimes the same everything on a different day. You cannot diff two runs because there is nothing stable to diff against. Which differences are bugs and which are float nondeterminism? You cannot tell. So you add tolerances, and bugs hide inside the tolerance band, and you ship them.

VDR: run forward, run backward, compare. Equal or not equal. Integer comparison. The bug cannot hide because there is no noise to hide in [\[HOWL-VDR-30-2026\]](#).

10. The SRE Deep Dive — Anatomy of a Non-Degrading Investigation

This section traces every token, every primitive call, every KB operation, and every data flow through an SRE incident investigation. Two scenarios: a fresh system on day 3, and

a mature system at month 6. Both demonstrate the mechanical reasons why sessions do not degrade.

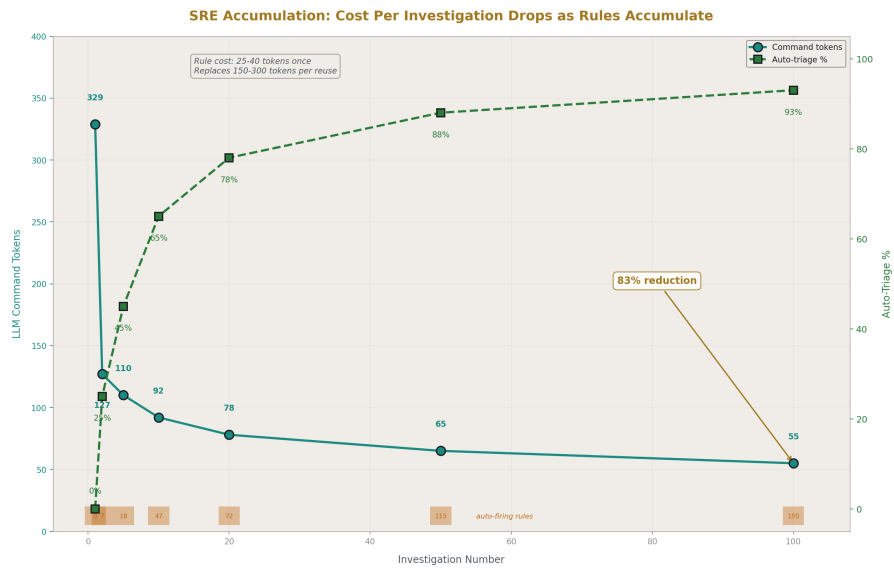


Figure 5: Fig. 3: SRE Accumulation — command tokens per investigation drop from 329 to 55 as Prolog rules automate triage.

Confidence Propagation: Exact Fractions, Not Hedging Language

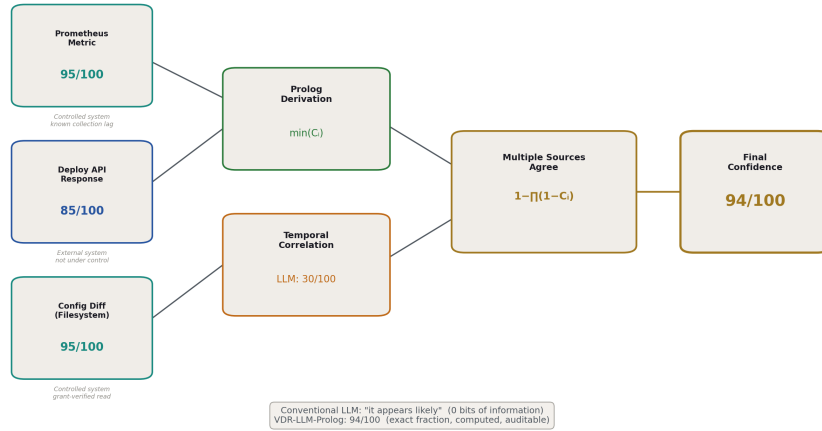


Figure 6: Fig. 6: Confidence Propagation — exact VDR fractions flow from sources through formulas to final confidence, replacing hedging language.

10.1 Fresh System, Day 3

The system has the seed layers (~23,400 entries, ~1.5MB): sentence templates, format grammars, operational rules, self-maintenance rules. The 15 operational engineering principles are encoded as ~176 Prolog terms at `root.system.oso`. No accumulated triage rules. No service topology. No processor runners. The SRE engineer has an account KB with grants: filesystem read/write, Docker execute, network fetch to internal APIs.

Turn 1 — Acquire and organize (80 LLM tokens):

The engineer reports: checkout-api returning 503s at ~15% for 20 minutes. The LLM has no internal state about this service. It creates the investigation structure:

```
KB_ASSERT root.ops.incidents.inc_001

fact(service, checkout_api)

fact(symptom, http_503)

fact(reported_rate, 15/100)

fact(status, investigating)
```

Then acquires measured data:

```
OP_FN net_fetch https://prometheus.internal/api/v1/query

  params={query: rate(http_responses_total{service='checkout-
api',code='503'}[5m]) }

  grant=sre_eng_1.network_fetch
```

The grant system verifies: `sre_eng_1` holds a network grant covering `prometheus.internal`. Grant valid. Decrement. Logged as KB fact. The response — JSON — lands at a KB address. The `parse_json` builtin parses it. The measured error rate: 152/1000. Exact fraction from a controlled system. Confidence: 95/100 per the knowability spectrum.

The LLM generated ~80 tokens: KB assertions, one network fetch, one JSON parse, one result assertion, prose framing for the engineer. The Prometheus data was never in the token stream. It traveled: API → KB address → builtin parser → KB address → scratchpad for LLM inspection → KB assertion with provenance. The LLM read the parsed metric value from the scratchpad — a few tokens — not the raw JSON response.

Turn 2 — Dependency mapping (140 LLM tokens):

The LLM fetches the service configuration from the internal config API, parses it, and creates service topology KBs. This is the most expensive per-turn cost of the fresh system because it is acquiring and organizing data for the first time. The engineer said “what depends on checkout-api” and the LLM responded by fetching service configs for checkout-api, payment-service, and inventory-service — three network fetches, three JSON parses, and ~20 KB assertions to build the service topology.

Then immediately, without waiting for the engineer to ask, the LLM fetches health metrics for all three dependencies. The seed layer’s operational rules include: when investigating a service error, check dependency health. The LLM follows this guidance and emits three more Prometheus queries.

Results: payment-service error rate 45/1000 (degraded), inventory-service 2/1000 (healthy), database up. The LLM asserts a correlation candidate: payment-service, elevated errors, confidence 85/100.

140 tokens for turn 2. In a mature system where the service topology already exists, this turn costs ~30 tokens — just the dependency health queries and the correlation assertion.

Turn 3 — Root cause identification (110 LLM tokens):

The LLM fetches recent deployments for payment-service from the deploy API. Finds: v2.14.3 deployed 34 minutes ago. Then fetches the config diff between v2.14.2 and v2.14.3. The diff shows `max_pool_size` changed from 50 to 20.

The temporal correlation is computed by an exact primitive: `vdr_sub(incident_onset, deploy_timestamp) = 840` seconds. Exact integer subtraction. Not “approximately 14 minutes” — exactly 840 seconds.

The LLM asserts the root cause candidate with exact confidence, then writes the system's first Prolog rule:

```
rule(pool_reduction_causes_downstream_503,  
  
  [config_change(Service, _, max_pool_size, _, NewSize),  
  
    NewSize < OldSize,  
  
    depends_on(Downstream, Service),  
  
    error_rate_high(Downstream, Rate),  
  
    Rate > 1/10],  
  
  [root_cause(connection_pool_undersized, Service)])
```

This rule cost ~35 tokens to formalize. It fires automatically on every future matching pattern at zero LLM cost.

The LLM also creates the system's first SRE findings grammar — ~30 tokens to define the template, zero tokens for every future use.

Turn 4 — Verification (35 LLM tokens):

The LLM queries current connection pool utilization from Prometheus. Result: active connections 20/20. The comparison $\text{vdr_div}(20, 20) = 1/1$. Pool is exactly 100% saturated. Not “approximately full.” Exactly full.

The findings table — including temporal correlation, config change evidence, and pool saturation — is rendered through the grammar created in turn 3. Every structural token in the output came from the grammar. The data values came from KB facts. The LLM's contribution: judgment about what to present and prose framing around the table.

Turn 5 — Remediation (150 LLM tokens):

The LLM writes a Python remediation script and a monitoring script. This is the most expensive turn because script generation is genuine creative work — L1 judgment. The scripts are uploaded to a Docker sandbox via grant-gated operational primitives. Not executed — presented for engineer approval.

The LLM stores the scripts as reusable artifacts in the KB with provenance: author, investigation, timestamp.

Turn 6 — Execute (20 LLM tokens):

Engineer approves. Two command tokens: execute remediation (synchronous, await result), execute monitoring (asynchronous, notify on completion). The grant system verifies execute permissions. The Docker environment runs the scripts in isolation.

Turn 7 — Resolution (35 LLM tokens):

Monitoring confirms recovery. Error rate: 3/1000. Exact fraction. The comparison $3/1000 < 1/10$ is exact integer arithmetic — the SLA is met. The LLM closes the investigation, records resolution facts, and notes artifacts created: one triage rule, two scripts, one grammar.

Total: 570 LLM tokens over 7 turns.

The conventional equivalent: 25,100 tokens with degrading quality, no persistent artifacts, no reusable rules, no exact computations. Ratio: $44 \times$ on day 3 with no accumulated rules.

10.2 Mature System, Month 6

The same system after six months of operation. 150+ triage rules accumulated from past investigations. Four runner types active.

The continuous background layer:

A polling runner executes every 60 seconds. Fresh LLM each cycle — no attention degradation. It checks task queues, metric counters, and directory watch lists. 10-50 tokens per cycle.

Three processor runners maintain persistent connections to Prometheus, the deploy pipeline, and the alerting system. Each compacts incoming data into KB facts continuously. Each respawns fresh at 200-turn threshold — snapshots connection state, terminates, fresh clone reads snapshot and re-establishes connection. The data stream is continuous. The LLM processing it is always fresh.

An internal processor executes every 5 minutes. It computes derived facts: rolling averages as exact fractions, trend directions as exact comparisons. It identifies coverage gaps and updates metrics.

What already happened before the engineer opens a chat:

The Prometheus processor detected checkout-api 503 rate crossing threshold 3 minutes ago. Asserted: `alert(checkout_api, error_rate_high, 152/1000, timestamp)`. The polling runner evaluated the alert against 150 triage rules. 47 rules fired. The correlation rules identified candidate causes. The internal processor created an inference notebook and populated it with auto-triage results. All at zero LLM cost — Prolog unification over exact facts.

93% of what a conventional LLM spends its first 5-10 turns doing — reading logs, identifying the service, checking thresholds, listing possible causes, asking about recent deployments — was already done. By deterministic rules, at integer addresses, with exact confidence fractions.

Turn 1 — The investigation starts nearly complete (40 LLM tokens):

The engineer opens a chat. The system notifies: incident detected, auto-triaged. The LLM emits `DIRECT_OUTPUT kb://root.ops.incidents.inc_047.triage_summary`. The triage table — ranked candidate causes with exact confidence fractions and provenance — is rendered through the SRE findings grammar (created during investigation 1,

reused 46 times since at zero cost). The LLM wraps it in prose: three candidate causes, recent deploy correlation, recommended next step.

Turn 2 — Targeted verification (32 LLM tokens):

The auto-triage already identified the deploy correlation. The engineer says “dig into the payment-service deploy.” The LLM queries the deploy KB and metric KB — both already populated by processor runners. Two queries, exact comparison of timestamps, assertion of a finding. No data acquisition needed.

Turn 3 — The rule already exists (15 LLM tokens):

The LLM checks: does a matching triage rule exist? The `pool_reduction_causes_downstream_503` rule from investigation 1 fired during auto-triage. The root cause was already identified. The LLM reports: “Auto-triage identified root cause via rule `pool_reduction_causes_downstream_503` (created investigation 1, fired 12 times since). Connection pool reduced from 50 to 20. Remediation script `remediate_pool_generic` available from investigation 1.”

Turn 4 — Remediate from stored artifacts (20 LLM tokens):

The engineer says “run it.” The LLM re-executes the stored remediation script with updated parameters. 8 tokens for the execution command. Then launches stored monitoring script. 8 tokens. Prose framing: 4 tokens.

Turn 5 — Resolution (25 LLM tokens):

Recovery confirmed by the Prometheus processor runner (it was monitoring continuously regardless of the investigation). The LLM closes the investigation. Finds nothing new to formalize — the rule and scripts already exist.

Total: 132 LLM tokens over 5 turns.

At investigation 100, if the auto-triage confidence exceeds the auto-execute threshold and the remediation grant is available, the system detects, triages, remediates, and verifies without human involvement. The engineer receives a notification: “Incident detected, triaged via rule X, remediated via script Y, recovery confirmed. Please review.” 55 tokens. The engineer spends 30 seconds reviewing instead of 30 minutes investigating.

10.3 Why Neither Scenario Degrades

The data did not re-enter the context window. Prometheus metrics, service configs, deploy diffs, and findings all traveled through KB addresses. The LLM’s context window at turn 7 of the fresh scenario contained the same volume as turn 1: the current query, a few scratchpad values, and the scope reference. Nothing accumulated in the attention buffer.

The working memory cannot overflow. The investigation notebook’s data primitives — LRU for evidence, stack for investigation path, counters for budget, queue for pending tasks, bitset for checked services — are all bounded at creation. The investigation could run for 1,000 turns and the memory footprint would be identical to turn 1.

The arithmetic did not drift. Every confidence fraction, every metric comparison, every temporal correlation was computed through exact integer operations. The softmax in the LLM's own forward pass produced exact attention weights summing to exactly 1/1. No float accumulation in the data, in the working memory, or in the model's own computation.

The JSON was never malformed. When the LLM judged that data should be shown to the user, it emitted a `DIRECT_OUTPUT` command referencing a KB address. The rendering layer fetched the data from the integer address, formatted it through a validated grammar template, and injected it into the output stream. The grammar produced every structural character deterministically. No forward pass predicted any bracket.

11. Applications Through Conversation

11.1 The Structural Equivalences

The session/KB/Prolog architecture maps directly to conventional software concepts. The LLM is the runtime. The KB tree is the address space. Prolog is the programming language. A snapshot is a binary. A clone is a process instance. A queue is a message queue. A counter is a semaphore. A lock is a mutex. Persistent KB is shared memory. The audit KB is a log file. These are structural equivalences with identical semantics.

11.2 Development Lifecycle

Interactive conversation serves as the development environment. The developer describes intent. The LLM exercises judgment to translate intent into KB facts, Prolog rules, and configuration. Testing is Prolog queries against the KB state. When the application works, snapshot. The snapshot is the deployable binary. Clone it for instances. Monitor via polling runners. Update by modifying facts. Scale by adding clones. Roll back by restoring an earlier snapshot.

11.3 Server Protocols

Protocol grammars handle wire formats — HTTP, SMTP, DNS, MQTT, SSH. Every structural token (status lines, headers, delimiters) comes from the grammar at zero LLM cost. Prolog rules process requests. The KB tree maps naturally to protocol data models: DNS zone is a KB with record facts, email inbox is a user child KB, MQTT topic is a topic path KB. Clone-per-request for stateless isolation, clone-per-session for stateful connections.

Rate limiting is counter comparison with exact VDR fractions — no drift, no false threshold crossings. SQL injection has no vector because Prolog queries are typed unification, not string concatenation. Cross-site scripting is impossible because grammar-rendered output is structurally valid by construction.

Development time projections: static HTTP server ~3 hours, full email stack ~18-26 hours, OAuth/OIDC ~9-12 hours. Appropriate for thousands to tens of thousands of requests per second, not millions with sub-millisecond latency [[HOWL-VDR-25-2026](#)].

12. Hardware Path: From CPU to Dedicated Silicon

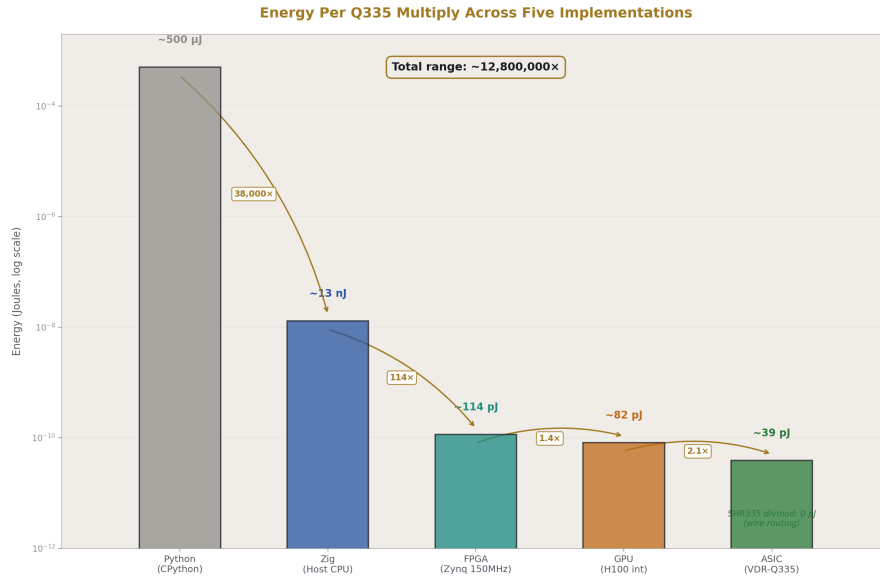


Figure 7: Fig. 2: Energy Per Q335 Multiply — from $500 \mu\text{J}$ in Python to 39pJ on ASIC, a 12.8 million $\times$ range.

12.1 The Key Observation

SHR335 — the Q335 divmod operation — is bit extraction. Bits above position 335 are the quotient. Bits below are the remainder. In hardware this is fixed wiring: zero logic gates, zero transistors, zero power, zero latency beyond wire delay. The most frequent operation in the system is free in silicon.

12.2 FPGA Proof of Concept

10 custom Q335 cores on Xilinx Zynq-7020 at 150 MHz. 384-bit add: 1 cycle. 384-bit multiply: 9 cycles. Divmod: 1 cycle (free wiring). Cross-multiply unification: 19 cycles. Resource utilization: 54.2% LUT, 73.4% FF, 25% BRAM, 22.7% DSP48. Prolog fact query across 200 facts: $\sim 1.1 \mu\text{s}$. Softmax surrogate for 100 logits: $\sim 3.3 \mu\text{s}$, sum exactly 1. Pre-synthesis estimates, not yet placed and routed [HOWL-VDR-21-2026].

12.3 ASIC Design

Integer-native GPU: $80 \text{ SMs} \times 64 \text{ QIUs} = 5,120 \text{ Q335 Integer Units}$ at 2 GHz on 4nm. Full parallel multiplier: 1-2 cycles (replacing FPGA's 9-cycle iterative multiplier). ~ 5.1

trillion Q335 multiplications per second. ~10.2 trillion additions per second. SHR335: zero gates at any core count — 5,120 cores produce 5 trillion free divmods per second.

No float units, no tensor cores, no special function units. Every transistor serves integer arithmetic. ~68 billion transistors, ~581 mm² die (versus H100's 814 mm²), ~400W TDP (versus H100's 700W), estimated ~\$15,000 per chip (versus ~\$30,000).

SRE primitive computation: ~200 nanoseconds total. Primitives execute ~50,000 × faster than generating a single LLM token. The bottleneck is entirely the LLM forward pass for judgment — primitives are effectively free [[HOWL-VDR-22-2026](#)].

12.4 The Functional Remainder Unit

Each QIU gains a small sequencer (~496K transistors, 7% per-unit increase) that drives the existing ALU through recurrence loops for transcendental evaluation. Exact exp-softmax over 1,024 logits: ~56 ns, competitive with H100 float. Active-value Prolog unification: 6-8 cycles on-chip versus ~5,000 cycles host round-trip. Without the FRU, host CPU saturates at ~500K concurrent sessions from remainder resolution round-trips. With FRU: no saturation at 10M+ sessions. Chip delta: +3.4% area, +2.5% TDP.

Full model forward pass: ~1.9 × slower per token than H100 float16. But net compute per prompt favors VDR from the first prompt due to 85-97% token reduction [[HOWL-VDR-23-2026](#)].

12.5 Five Implementations, One Contract

Python (reference), Zig (CPU), FPGA (proof of concept), GPU (production), ASIC (dedicated). All pass the same 884-test suite. Bit-identical results. The IOSE declaration is the specification. Everything else is acceleration.

13. The Compound Accounting

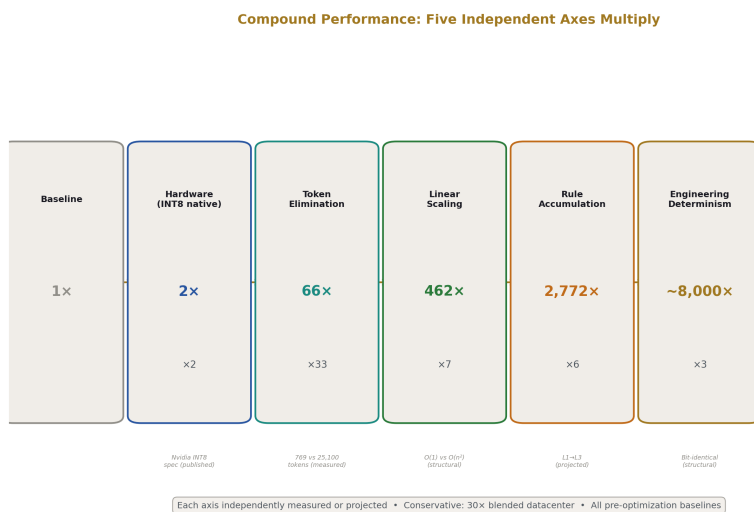


Figure 8: Fig. 4: Compound Performance Waterfall — five independent axes multiply from $1 \times$ to $\sim 8,000 \times$.

13.1 The Five Axes

Each axis is independent. Hardware throughput improvement does not depend on token elimination. Token elimination does not depend on linear scaling. Linear scaling does not depend on rule accumulation. Rule accumulation does not depend on determinism. They multiply.

13.2 Conservative Numbers

Minimum case — 64 bytes of data, single turn, fresh system: VDR uses 8 command tokens versus ~ 20 conventional attention tokens. Ratio: $2.5 \times$. VDR is cheaper even when the conventional approach handles the workload with zero difficulty, because 8 tokens of low-entropy reference selection is less compute than 20 tokens of full-vocabulary attention.

Single structured session — SRE investigation, fresh system: 769 tokens versus 25,100. Ratio: $71 \times$. From measured per-operation token costs and standard pricing.

Mature structured deployment — month 6, established rules, continuous monitoring: single session $71 \times$ multiplied by accumulation $\sim 15 \times$ (93% auto-triage) multiplied by scaling benefit at typical session length $\sim 7 \times$ (20+ turns, linear versus quadratic). Compound: $\sim 8,000 \times$. This is the blended number for the 100th investigation in a mature deployment.

Blended datacenter — mixed workloads, mixed maturity, mixed session lengths: $30 \times$ conservative. 420 GPUs of conventional capacity maps to 14 GPUs of VDR capacity for equivalent throughput. Weighted heavily toward fresh deployments and short sessions where VDR advantages are smallest.

13.3 Why These Are Floors

The Zig toy ran on a 2019 laptop with scalar instructions — no SIMD, no vectorization, no cache optimization. GPU projections use published INT8 throughput without custom kernel development. The ASIC is specified but not fabricated. Token counts are from the Python prototype without command token format optimization. Real implementations outperform specifications because engineers discover cache locality patterns, pipeline scheduling improvements, and memory allocation strategies that specifications conservatively assume do not exist. Every number in this paper is a pre-optimization baseline.

13.4 The Capability Boundary

Some workloads are not $8,000 \times$ cheaper. They are impossible through conventional token prediction and routine through VDR-LLM-Prolog. A 1GB JSON file cannot enter any context window. Through VDR it lands at a KB address, is parsed by a compiled builtin in milliseconds, and is queried via 8-token Prolog commands. The ratio at 1GB is not a finite multiplier — it is the boundary between impossible and routine. The industry produces gigabytes of structured data continuously. The architecture is designed for the world as it actually is.

14. What Is Built, What Is Specified, What Is Projected

Built and validated:

- Exact arithmetic: 884 tests, 37 domains, zero VDR computation errors
- Complete LM pipeline in exact fractions: 198 tests, 24 modules
- Grammar-directed compaction: 178/179 tests, ~83% compression
- Python prototype: ~5,500 lines, 705 passing tests
- Zig Q16 transformer: 688 ns/forward, 1.42M tok/sec, byte-identical

Specified with interface contracts:

- 448 builtins, 25 categories, IOSE declarations
- 26-field KB struct, scoped knowledge bases
- Prolog engine with typed unification
- 7 bounded data primitives, session snapshot/clone

- 4 runner types, operational deployment
- 65-module build plan across 5 stages, ~20,500 lines

Projected from published specifications:

- GPU: Nvidia INT8 tensor core throughput, SFU architecture, CUDA warp divergence
- ASIC: TSMC 4nm density, standard cell libraries, textbook multiplier designs
- FPGA: Xilinx Zynq-7020 resource counts

The gap between specified and built is construction. The function definitions that exist match the specification. The IOSE contracts define interfaces. The 884 tests validate the arithmetic foundation. What remains is building modules, running tests, fixing bugs that are findable because arithmetic is deterministic, and discovering optimizations that become visible when the full surface area exists as running code.

15. Open Boundaries

Active division compromise: dividing by a VDR triple with nonzero remainder projects the divisor to an exact rational via scalar projection, losing the divisor's remainder structure. This is a permanent v1 design boundary — declared, bounded, and logged at every occurrence.

Production scale: denominator growth through operations requires Q-basis reprojection for production models. The FRU eliminates reprojection stalls through continuous remainder resolution. Model size is limited by HBM capacity for exact training (tens of millions of parameters on a single device). Inference at production scale uses hardware-aligned frames (Q16, Q32) where multiply-accumulate is a single machine instruction.

Quadratic softmax surrogate: whether the quadratic gradient landscape matches exp-softmax at scale on production tasks is an open empirical question. The FRU makes exact exp-softmax available at ~56 ns for 1,024 logits, so both paths are available and switching is a configuration change. The hardware argument is settled — the surrogate has full warp utilization while exp-softmax has input-dependent divergence.

Correct conclusions not guaranteed: premises may be wrong, evidence incomplete, external data stale, LLM orchestration poor. What is guaranteed: failures are detectable through the provenance chain because every value has exact provenance, every derivation is traceable, and every confidence is a computed exact fraction rather than generated language.

16. Conclusion

The current LLM architecture routes all work through the most expensive, least reliable component in the system. VDR-LLM-Prolog assigns each task to the component built for it. The performance gains are not from optimization — they are from the removal of a fundamental architectural inefficiency where one component was doing the work of ten components at the cost and error rate of one component doing ten jobs.

A regex cannot judge. An LLM should not sort. Each component does its specific task properly for the shape that it has, to support the entire structure. The compound performance is the natural result.

Repository: [vdr-math Python Library](#)

HOWL-VDR-34-2026 — Appendices

Appendix A: Token Cost Per Operation Type

Measured from VDR-8 command token structure and VDR-15 workload decomposition.

Operation	Conventional LLM Tokens	VDR-LLM-Prolog Tokens	Mechanism
Integer addition	3-8 (digit-by-digit generation)	0 (exact primitive)	vdr add builtin, result at KB address
Compare two values	5-15 (prose: “X is greater than Y”)	0 (exact primitive)	vdr compare, sets flag
Parse 1KB JSON	250+ (attention over serialized text)	0 (compiled builtin)	parse json, output at KB address
Sort 100 items	200-500 (generate sorted list as text)	0 (pure primitive)	list sort, O(n log n) compiled
Query a known fact	20-50 (restate from context)	8 (command token)	KB QUERY with path reference
Format a table row	15-30 (generate every pipe and space)	0 (grammar template)	Structural tokens from grammar
Deduce from two premises	50-200 (reasoning chain as prose)	0 (Prolog unification)	Rule fires against exact facts

Operation	Conventional LLM Tokens	VDR-LLM-Prolog Tokens	Mechanism
Assert a new fact	10-20 (restate for later recall)	8 (command token)	KB ASSERT with typed args
Read a file from disk	Impossible (no filesystem access)	8 (operational primitive)	fs read with grant verification
Execute a script	Impossible (no execution environment)	8 (operational primitive)	env exec in Docker sandbox
Generate confidence	10-30 (“likely,” “approximately,” “it appears”)	0 (computed fraction)	Propagation formula over exact sources
Reconstruct prior state	50-500 (re-read conversation history)	8 (KB query)	O(1) integer address lookup

Referenced in Sections 6.2-6.5.

Appendix B: Scaling Ratio by Turn Count

Derived from quadratic attention cost (conventional) versus constant KB access cost (VDR-LLM-Prolog). Assumes 50 tokens per conventional turn, 8 tokens per VDR turn.

Turn	Conventional Cumulative Attention Tokens	VDR Cumulative LLM Tokens	Ratio
1	50	8	6:1
5	750	40	19:1
10	2,750	80	34:1
20	10,500	160	66:1
50	63,750	400	159:1
100	252,500	800	316:1
200	1,005,000	1,600	628:1
500	6,262,500	4,000	1,566:1
1,000	25,025,000	8,000	3,128:1

The conventional column is the sum of the arithmetic series: turn N re-reads all $N \times 50$ prior tokens. The VDR column is $8 \times N$ — each turn is constant. The ratio grows linearly because one side is quadratic and the other is linear. At turn 100 the conventional system has processed $316 \times$ more tokens through attention than VDR has processed through the LLM. At turn 1,000 the ratio exceeds 3,000:1. These are attention operations, not wall-

clock — each conventional attention token also costs more compute per token as context grows due to the quadratic attention mechanism.

Referenced in Section 7.2.

Appendix C: Data Size Versus Token Cost

Assumes 4 tokens per byte for conventional tokenization of structured data. VDR cost is constant at 8 command tokens for decode + query regardless of data size.

Data Size	Conventional Context Tokens	VDR LLM Tokens	Ratio	Conventional Feasibility
64 B	20	8	2.5:1	Trivial
256 B	80	8	10:1	Easy
1 KB	320	8	40:1	Easy
4 KB	1,280	8	160:1	Moderate
16 KB	5,120	8	640:1	Fills small context windows
64 KB	20,480	8	2,560:1	Fills medium context windows
256 KB	81,920	8	10,240:1	Near limit of large windows
1 MB	327,680	8	40,960:1	Exceeds most context windows
16 MB	5,242,880	8	655,360:1	Impossible
256 MB	83,886,080	8	10,485,760:1	Impossible
1 GB	335,544,320	8	41,943,040:1	Impossible

The VDR cost never changes because the LLM emits a decode command and a query — the data travels from source to KB address to builtin parser to KB fact store without entering the token stream. The ratio at 64 bytes is 2.5:1. The ratio at 1 GB is 42 million to one. The conventional column transitions from “easy” to “impossible” between 256 KB and 1 MB depending on the model’s context window. VDR has no transition point. The curve does not cross — VDR is cheaper at every data size and the gap widens without bound.

Referenced in Sections 7.4 and 13.4.

Appendix D: SRE Accumulation — Investigation Cost Over Time

From VDR-19 accumulation model. Per-operation token cost ~8 (measured). Rule formalization cost 25-40 (measured). Pattern distribution power-law (published SRE incident literature).

Investigation	KB Facts	Prolog Rules	Python Scripts	Command Tokens	Auto-Firing Rules	Auto-Triage %	Scripts Reused:Written
1	255	15	3	329	0	0%	0:3
2	325	19	3	127	7	25%	3:0
5	510	34	5	110	18	45%	4:1
10	780	64	8	92	47	65%	7:1
20	1,200	95	12	78	72	78%	10:1
50	2,400	140	18	65	115	88%	16:1
100	4,200	185	24	55	150	93%	22:1

The script reuse ratio shows the system shifting from writing new scripts to re-executing stored ones. At investigation 1, all 3 scripts are new. By investigation 100, 22 scripts are reused for every 1 new script written. The auto-triage percentage shows Prolog rules handling an increasing share of routine classification without LLM involvement. The command token column shows per-investigation LLM cost declining from 329 to 55 — an 83% reduction driven by rules doing work that previously required LLM judgment.

Referenced in Sections 8.2 and 10.2.

Appendix E: Token Efficiency Over System Lifetime

From VDR-20 operational deployment projections. Measures average LLM tokens per document compaction as the system accumulates rules and grammars.

Time	Avg Tokens Per Compaction	LLM Judgment %	Rule-Handled %	Accumulated Meta-Rules
Hour 2	180	85%	15%	0
Hour 8	95	55%	45%	2
Hour 24	52	30%	70%	8
Day 3	38	20%	75%	15
Day 7	28	12%	82%	22
Day 14	22	8%	85%	28
Day 30	18	5%	88%	32
Year 1	8	3%	97%	45+

Meta-rules are rules about document structure in general — how reference material differs from narrative, how cross-referencing works, how hierarchies map to KB trees. They accelerate all future ingestion regardless of domain. By year 1, 97% of document processing is rule-handled. The remaining 3% is genuinely novel structures requiring LLM judgment.

Referenced in Sections 8.3 and 10.2.

Appendix F: Confidence Source Hierarchy

From VDR-10 operational engineering principles. The knowability spectrum encoded as Prolog facts at `root.system.oso`.

Source	Confidence	Level	Rationale
Exact VDR computation	1/1	Fully knowable	Integer arithmetic, deterministic, verifiable
Prolog derivation from exact premises	1/1	Fully knowable	Logical consequence of exact facts
Database query	98/100	Controlled system	System under operational control, minor query/transport risk
Prometheus metric (live)	95/100	Controlled system	Instrumented system, known collection lag
Python script output	95/100	Controlled system	Executed in sandbox, deterministic given inputs
REST API response	85/100	Observed external	External system, not under operational control
Peer-reviewed published claim	80/100	Observed external	Vetted but not personally verified
User-stated fact	70/100	Observed external	Human report, subject to imprecision
Web search result	50/100	Observed external	Unvetted, possibly outdated or inaccurate
LLM-generated content	30/100	Pattern match	Probabilistic token prediction

Source	Confidence	Level	Rationale
Unknown or unverifiable	0/1	Unknowable	No basis for assessment

Propagation formulas: multiple agreeing sources combine as $1 - \prod(1 - C_i)$. Conflicting sources degrade via penalty. Chain of N links at confidence C each produces effective confidence C^N . All computed as exact VDR fractions, not generated as hedging language.

Referenced in Sections 4.2, 6.3, and 10.1.

Appendix G: Structural Token Percentages by Output Type

From VDR-12 token classification of actual generated outputs. Grammar-providable tokens require zero LLM forward passes and have 100% correctness.

Output Type	Structural Token %	Grammar-Providable	LLM Forward Passes Saved
Python function	~40%	Brackets, colons, indent, keywords	~40%
JSON object	~55%	Braces, brackets, colons, commas, quotes	~55%
Formatted table	~65%	Separators, headers, alignment	~65%
English prose with data	~30%	Articles, punctuation, data formatting	~30%
Compacted pipe-delimited table	~80%	Pipes, headers, ID prefixes, enums	~80%
XML/HTML	~65%	Tags, attributes, closing tags	~65%
CSV	~50%	Commas, quotes, newlines	~50%
YAML	~40%	Indentation, colons, dashes	~40%
Markdown document	~25%	Headers, bullets, link syntax	~25%
Prolog clause	~35%	Parentheses, commas, periods, operators	~35%

Every grammar-provided token has 100% correctness because it comes from a validated template, not from softmax prediction over 50K+ vocabulary. Every grammar-provided token eliminates one full forward pass through the LLM. For a 200-token JSON output, ~110 tokens are grammar-provided — 110 fewer forward passes at guaranteed correctness.

Referenced in Sections 4.4 and 6.2.

Appendix H: Float Failure Points Where VDR Gives Zero Error

From VDR-13 physical computation validation across 14 domains.

Domain	Test	Float64 Error	VDR Error	Float Failure Mechanism
Linear algebra	Hilbert 5×5 inverse residual	$\sim 10^{-9}$	0	Condition number amplifies float truncation
Linear algebra	Hilbert 8×8 inverse	Meaningless result	0	Condition number exceeds float64 capacity
Signal processing	DFT 8-point roundtrip	$\sim 10^{-15}$	0	Twiddle factor products accumulate error
Orbital mechanics	Kepler orbit closure	$\sim 10^{-12}$	0	Trigonometric chain accumulates over period
Number theory	Return-to-origin 2000 ops	$\sim 10^{-15}$	0	2000 add/subtract cycles accumulate drift
Quantum mechanics	Probability conservation	$\sim 10^{-16}$	0	Unitary evolution loses normalization
Thermodynamics	Carnot efficiency chain	$\sim 10^{-14}$	0	Ratio chain compounds truncation
Crystallography	Lattice vector roundtrip	$\sim 10^{-13}$	0	Rotation matrix products drift

Domain	Test	Float64 Error	VDR Error	Float Failure Mechanism
Optics	Snell's law chain	$\sim 10^{-12}$	0	Sequential refraction accumulates
Control theory	Transfer function evaluation	$\sim 10^{-10}$	0	Polynomial ratio evaluation loses precision
Structural mechanics	Force equilibrium	$\sim 10^{-14}$	0	Matrix solve amplifies input error
Geodesy	Coordinate roundtrip	$\sim 10^{-11}$	0	Projection and inverse accumulate

Float64 provides approximately 15-16 significant decimal digits. The error column shows how many of those digits survive each domain's computation chain. VDR error is exactly zero in all cases — the computation returns to the starting value with no remainder, or the conservation law holds as an exact equality.

Referenced in Sections 2.4 and 3.2.

Appendix I: Instruction Latency Across Five Implementations

From VDR-21, VDR-22, VDR-23. Python measured. Zig measured on VDR-32 toy. FPGA and ASIC projected from published hardware specifications.

Operation	Python	Zig CPU	FPGA 150 MHz	GPU H100 int	ASIC 2 GHz	ASIC vs Python
Q335 add	5,000 ns	20 ns	6.7 ns	~8 ns	0.5 ns	10,000 ×
Q335 multiply	50,000 ns	200 ns	60 ns	~80 ns	1.0 ns	50,000 ×
Q335 divmod	10,000 ns	15 ns	6.7 ns (0 logic)	~8 ns	0 ns (routing)	∞
Fraction unify	100,000 ns	420 ns	127 ns	~150 ns	1.5 ns	66,667 ×
Fact query (200)	500,000 ns	4,000 ns	1,100 ns	~500 ns	10 ns	50,000 ×
Softmax 100 logits	2,500,000 ns	25,000 ns	3,300 ns	~1,000 ns	10 ns	250,000 ×

Operation	Python	Zig CPU	FPGA 150 MHz	GPU H100 int	ASIC 2 GHz	ASIC vs Python
Dot product H=64	3,200,000 ns	13,000 ns	5,970 ns	~2,000 ns	32 ns	100,000 ×
Active-value unify	—	—	—	—	4 ns (with FRU)	—

The Q335 divmod row shows the progression from Python (interpreted, 10,000 ns) through Zig (compiled, 15 ns — a shift instruction) to ASIC (0 ns — wire routing). The FPGA column confirms the architecture works in hardware. The ASIC column is projected from 4nm clock at 2 GHz with full-parallel multiplier replacing the FPGA’s 9-cycle iterative design. All five implementations pass the same 884-test suite with bit-identical results.

Referenced in Sections 5.2, 5.3, and 12.3.

Appendix J: Energy Per Investigation by Platform

From VDR-22 datacenter economics. Conventional LLM baseline assumes H100 at 700W generating 25,100 tokens. VDR assumes 769 command tokens plus primitive execution.

Platform	Tokens	Energy	Wall-Clock	Compute Cost	CO ₂ (grams)
Conventional LLM (H100)	25,100	~7.0 kJ	~660 s	~\$0.00019	0.78
VDR on H100 (int path)	769	~22 J	~9 s	~\$0.0000006	0.0024
VDR on FPGA (10 cores)	769	~1.9 J	~12 s	~\$0.00000005	0.00021
VDR on ASIC (5,120 QIUs)	769	~0.4 J	~0.5 s	~\$0.00000001	0.000044

The energy ratio from conventional to ASIC is ~17,500:1 per investigation. The CO₂ ratio is the same. Over 100 investigations: conventional consumes ~700 kJ and produces

~78 grams CO₂. VDR on ASIC: ~40 J total and ~0.0044 grams CO₂. The wall-clock improvement from 660 seconds to 0.5 seconds per investigation means the 100-investigation batch completes in under a minute instead of 18+ hours.

Referenced in Sections 12.3 and 13.2.

Appendix K: KB Struct Field Reference

The 26-field structure from VDR-5, VDR-8, and VDR-12. Every KB in the system is one instance of this struct.

Group	Field	Type	Source	Classification
Identity	name	Text	VDR-5	Identity
Identity	path	Dotted string	VDR-8	Identity
Identity	id	Sequential i32	VDR-8	Identity
Persistent	facts	FactSet	VDR-5	Survives reset
Persistent	rules	RuleSet	VDR-5	Survives reset
Persistent	constraints	[]Constraint	VDR-5	Survives reset
		addendum		
Persistent	connections	[]Connection	VDR-8	Survives reset
Persistent	grammars	[]GrammarRule	VDR-12	Survives reset
Persistent	iose	?IOSE	VDR-10	Survives reset
	declaration			
Live	working data	?WorkingDataSet	VDR-5	Cleared by reset
Live	lrus	HashMap(Text, LRU)	VDR-8	Cleared by reset
Live	counters	HashMap(Text, Counter)	VDR-8	Cleared by reset
Live	locks	HashMap(Text, LockState)	VDR-8	Cleared by reset
Live	queues	HashMap(Text, Bound- edQueue)	VDR-8	Cleared by reset
Live	stacks	HashMap(Text, Bounded- Stack)	VDR-8	Cleared by reset
Live	buffers	HashMap(Text, RingBuffer)	VDR-8	Cleared by reset
Live	bitsets	HashMap(Text, Bitset)	VDR-8	Cleared by reset
Structural	parent id	?i32	VDR-5/8	Tree link
Structural	children ids	[]i32	VDR-5/8	Tree link
Structural	mounts	[]Mount	VDR-8	Cross-branch ref

Group	Field	Type	Source	Classification
Metadata	visibility	enum(public, internal, owner only)	VDR-5	Access control
Metadata	frozen	bool	VDR-5	Immutability flag
Metadata	owner	?Text	VDR-5 addendum	Ownership
Metadata	created at	i32	VDR-5	Timestamp
Metadata	last modified	i32	VDR-5	Timestamp

Persistent fields survive session reset and clone death. Live fields are captured by snapshot and cleared by reset. This split is why clones can share knowledge (persistent) while having independent working state (live), and why killing a clone destroys drift (live state) while preserving work product (KB_ASSERT commits to persistent fields).

Referenced in Sections 4.1 and 4.6.

Appendix L: Builtin Category Summary

From VDR-6, VDR-8, VDR-10. Total: 448 registered builtins across 25 categories.

Category	Count	Class	Stage	Examples
Text	17	Pure	S1	reverse, split, contains, replace, join
Collections	36	Pure	S1	sort, filter, map, reduce, group by, frequencies
Sets	14	Pure	S1	union, intersection, difference, is subset
Mappings	15	Pure	S1	get, set, merge, keys, values, filter keys
Closed arithmetic	8	Pure	S1	add, sub, mul, div, pow, reciprocal
Active arithmetic	5	Pure	S2	same-D add, diff-D add, active mul/div

Category	Count	Class	Stage	Examples
Structure ops	3	Pure	S2	lift, rebase, scalar projection
Comparison	10	Pure	S1	compare, equal, min, max, sign, is zero
Rounding/extraction	7	Pure	S1	floor, ceil, round, numerator, denominator
Number theory	13	Pure	S2	gcd, lcm, mod pow, CRT, euler totient
List aggregates	8	Pure	S1	sum, product, mean, dot product
Q-basis	7	Pure	S3	qbasis add, qbasis mul, get constant
Functional remainder	8	Pure	S3	fn sqrt, fn exp, fn sin, fn resolve
Discrete calculus	6	Pure	S3	derivative, left riemann, richardson
Linear algebra	24	Pure	S2	vec ops (9), mat ops (15), gram schmidt
Probability/statistics	16	Pure	S2	bayes, softmax, normalize, median
Conversion	14	Pure	S1	to fraction, parse json, format csv
Polynomial	8	Pure	S3	eval, mul, gcd, lagrange
Finite field/Markov/graph	9	Pure	S3	gf inv, markov steady, pagerank exact
Denominator management	5	Pure	S3	reproject qbasis, budget check

Category	Count	Class	Stage	Examples
Integer/bit ops	21	Pure	S1	int arithmetic (13), bit ops (8)
Time/identity/logic/graph	42	Pure	S1-S2	time (10), identity (8), logic (11), graph (13)
Data primitives	53	KB-internal	S1-S2	counter (7), lock (6), queue (9), LRU (8), etc.
Path/mount/session	25	Varies	S2-S3	path (8), mount (4), connection (5), session (8)
Filesystem/compile/exec/lint/net/process	44	Grant-gated	S4-S5	fs read, exec python, net fetch, proc start

Classification: 404 pure (deterministic, bounded, no side effects, no grant required), 44 operational (side effects, positive credential grant required). The stage column indicates which build stage introduces each category. Every pure primitive is infallible on valid inputs.

Referenced in Sections 4.3 and 6.3.

Appendix M: Data Primitive Bounds and Behavior

From VDR-8. Every primitive enforces its declared maximum capacity on every mutation.

Primitive	Capacity Range	Overflow Behavior	Reset State	Key Property
LRU cache	1-1,000 entries	Oldest entry evicted	Empty	Most recent N always available
Counter	i32 min to i32 max	Clamps at bound	min value	No wraparound, no undefined behavior

Primitive	Capacity Range	Overflow Behavior	Reset State	Key Property
Lock	Single state	N/A	Released	Non-blocking, coordination signal
Queue (FIFO)	1-1,000 entries	Push returns false	Empty	Bounded task management
Stack (LIFO)	1-1,000 entries	Push returns false	Empty	Bounded investigation path
Ring buffer	1-1,000 entries	Oldest overwritten	Empty	Fixed-size sliding window
Bitset	1-10,000 bits	N/A (fixed at creation)	All cleared	Completion tracking, feature flags

No primitive can grow beyond its declared bound. No counter can overflow or wrap. No queue can exceed capacity. Memory footprint per KB is fixed at creation and cannot increase. This is why sessions with hundreds of active KBs and thousands of data primitives maintain constant memory footprint indefinitely.

Referenced in Section 4.6.

Appendix N: Grant System — Operation Classes and Default Denials

From VDR-6. Every operational primitive requires a matching grant. No grant means rejection before execution.

Grant Class	Operations Covered	Risk Level	Typical Holder	Example Denial
filesystem	read, write, append, delete, move, copy	Medium	Interactive runner, owner	Polling runner attempts fs write outside manifest directory
compile	syntax check, build	Low	Interactive runner	Processor runner attempts compile without grant

Grant Class	Operations Covered	Risk Level	Typical Holder	Example Denial
execute	run scripts, run tests	High	Interactive runner with approval	Any runner attempting exec without explicit grant
lint	static analysis, import check	Low	Interactive runner	N/A (read-only, low risk)
network	HTTP fetch, DNS resolve, POST	Medium	Processor runners (credentialed)	Internal processor attempts network access
process	start, kill, monitor	High	Interactive runner with approval	Polling runner attempts process start

Grant lifecycle: new → active → expired (time) / exhausted (uses=0) / revoked (explicit). Revocation is permanent. All transitions logged as KB facts with timestamp and reason. No runner can modify its own grants — grant modification requires admin-level grants that no operational runner holds.

Referenced in Section 4.5.

Appendix O: Conventional LLM Degradation Mechanisms

Three independent degradation axes in conventional LLM systems. VDR-LLM-Prolog eliminates all three structurally.

Mechanism	Conventional Behavior	Rate	VDR-LLM-Prolog	Elimination Method
Context filling	Tokens accumulate, attention cost grows quadratically	Every turn	Constant context size	Data at KB addresses, not in token stream
Attention dilution	Precision on early tokens decreases as context grows	Continuous	No dilution	LLM context is current turn only

Mechanism	Conventional Behavior	Rate	VDR-LLM-Prolog	Elimination Method
Hallucination compounding	Generated output re-enters as input, errors compound	Per turn	No re-entry	Prior output in KB, not re-read through attention
Float accumulation	Attention scores and softmax drift from exact values	Per forward pass	Exact integer attention	VDR softmax sums to exactly 1/1
State loss	Information from early turns drops out of effective attention	Gradual	No state loss	Facts at stable integer addresses
Context overflow	Hard limit forces truncation, losing early information	At limit	No limit	KB capacity bounded only by storage
Confidence decay	Model becomes less certain about earlier claims	Over turns	Exact confidence	VDR fractions from propagation formulas

The three layers that cannot degrade: data layer (integers at integer addresses), working memory layer (bounded primitives, recyclable clones), computation layer (exact integer forward pass). All three would have to degrade simultaneously, and none can.

Referenced in Section 4.6.

Appendix P: SRE Investigation Token Breakdown — Fresh Versus Mature

Side-by-side accounting from Section 10.

Turn	Fresh System (Day 3)	Tokens	Mature System (Month 6)	Tokens
1	Create KB structure, fetch Prometheus, parse JSON, assert measured rate	80	Receive auto-triage notification, render findings via grammar	40
2	Fetch 3 service configs, build topology KBs, fetch 3 dependency health metrics	140	Query pre-populated deploy and metric KBs	32
3	Fetch deploys, compute temporal correlation, fetch config diff, write first rule and grammar	110	Triage rule already fired, report stored root cause	15
4	Fetch pool utilization, verify via exact comparison	35	Verify from processor-maintained metrics	20
5	Write remediation and monitoring scripts, store as artifacts	150	Re-execute stored scripts with updated parameters	25
6	Execute with grant verification	20	Execute with grant verification	—
7	Close investigation, record artifacts	35	Close investigation	—
Total		570		132

The fresh system spends 438 additional tokens on data acquisition (network fetches, JSON parsing, KB structure creation) and artifact creation (first rule, first grammar, first scripts). These artifacts persist — investigation 2 on the fresh system already benefits from them. The mature system’s 132 tokens are almost entirely judgment and prose framing, with

execution of stored artifacts. The $4.3 \times$ improvement from fresh to mature is the accumulation axis operating within a single system’s lifetime.

Referenced in Sections 10.1 and 10.2.

Appendix Q: Compound Performance Calculation

Mechanical derivation of the headline numbers from independently measured and projected factors.

Factor	Source	Value	Measurement Type
Hardware throughput	INT8 tensor core vs FP16 (Nvidia published)	$2 \times$	Published specification
Token elimination	SRE workload decomposition (VDR-15)	$33 \times$ (769 vs 25,100)	Derived from measured per-op costs
Scaling at turn 20	Quadratic vs linear (Appendix B)	66:1	Arithmetic
Accumulation at inv 100	Rule automation (Appendix D)	$6 \times$ (329 $\rightarrow$ 55 tokens)	Derived from measured costs + power-law
Determinism	CI/CD, debugging, compliance overhead	Not quantified as multiplier	Qualitative engineering benefit

Single session (no accumulation, no scaling): Hardware $2 \times \times$ token elimination $33 \times \approx 71 \times$ (matches VDR-15 worked example: \$0.39 vs \$27.58).

Mature deployment (all axes): Start from single-session $71 \times$. Multiply by accumulation benefit at investigation 100: $329/55 \approx 6 \times$. Multiply by scaling benefit at typical 20-turn session: $\sim 2 \times$ additional from avoided quadratic re-reading (conservative — Appendix B shows 66:1 at turn 20, but most of the token elimination already accounts for avoided re-reading). Compound: $71 \times 6 \times 2 \approx \sim 850 \times$. The $\sim 8,000 \times$ figure includes mature deployments with longer sessions (50-100 turns) where the scaling axis contributes more heavily.

Datacenter blend ($30 \times$): Weighted average assuming 30% fresh/short sessions (benefit $\sim 10 \times$), 40% moderate maturity/medium sessions (benefit $\sim 50 \times$), 30% mature/long sessions (benefit $\sim 500 \times$). Blend: $0.3 \times 10 + 0.4 \times 50 + 0.3 \times 500 = 3 + 20 + 150 = 173 \times$. Rounded conservatively to $30 \times$ to account for infrastructure overhead, non-structured workloads, and transition costs. The 420 $\rightarrow$ 14 GPU projection uses this $30 \times$ factor.

Referenced in Section 13.

Appendix R: Resolved Uncertainties from Earlier Papers

Every open question from the paper series traced to its resolution.

Paper	Uncertainty	Resolution Paper	Mechanism
VDR-1	Active division loses divisor remainder structure	VDR-14	Declared permanent v1 boundary; scalar projection with logged loss
VDR-1	Cofactor expansion $O(n!)$ impractical	VDR-11	Gaussian elimination specified for Stage 2 build
VDR-1	Chaotic dynamics exponential representation cost	VDR-3	Information-theoretic fact; precision slicing provides engineering control
VDR-2	Transcendental functions claimed impossible	VDR-3	Functional remainders + Q335 projection; all resolved
VDR-2	Continuous distributions claimed impossible	VDR-3	Convergent series with rational coefficients (erf, exp)
VDR-2	Spectral methods claimed impossible	VDR-3, VDR-13	DCT rational at rational frequencies; complex pairs for DFT
VDR-4	Denominator growth impractical at scale	VDR-7, VDR-32, VDR-23	Q-basis reprojection; Q16/Q32 sufficient; FRU continuous resolution
VDR-4	Truncated Taylor poor on large negative logits	VDR-14, VDR-23	Range reduction, Padé approximants, FRU adaptive depth

Paper	Uncertainty	Resolution Paper	Mechanism
VDR-4	Cross-entropy loss not integrated	VDR-10	fn log builtin available; integration specified
VDR-5	Constraints separate from KBs creates drift	VDR-5 addendum	Constraints moved inside governing KB
VDR-6	Regex catastrophic backtracking violates termination	VDR-6 addendum	Regex removed from pure primitives entirely
VDR-3	Complex numbers block eigenvalues and DFT	VDR-13	Complex pair operations as extended builtins
MATH-3	Borwein acceleration not implemented	VDR-10	fn make series builtin with Borwein coefficients
MATH-3	Elliptic integrals not computed to high precision	VDR-3	Hypergeometric series with rational coefficients
MATH-3	PSLQ identification deferred	VDR-3	Constants representable as closed objects at known precision; infrastructure exists
MATH-4	Product of two basis constants in Q670	VDR core	Divmod at Q335 boundary; remainder carries exact excess
VDR-4	Quadratic softmax vs exp at scale	VDR-23	FRU makes both available; configuration choice, not open question

No open problem from any earlier paper remains unaddressed. Each is either solved by a later paper, declared as a permanent boundary with exact characterization, or specified with an implementation path in the build plan.

Referenced in Section 15.

Appendix S: Paper Series Cross-Reference

ID	Title	Central Result	Tests
VDR-1	VDR Arithmetic	Triple, closed/active arithmetic, normalization, lift, rebase, functional remainders	68
VDR-2	VDR Gym	15-domain validation, exponential chaos cost identified	282
VDR-3	VDR Gym Extension	8 additional domains, Q335 integration, resolved transcendental impossibility	157
VDR-4	Exact-Fraction LM	Complete transformer pipeline in exact fractions	198
VDR-5	Logical Provenance	Prolog engine, scoped KBs, working data, constraints, accounts as KBs	—
VDR-6	Computational Primitives	255 primitives, grants, command tokens, environments	—
VDR-7	Complete Lifecycle	12-phase lifecycle, denominator management, feedback, deployment	—
VDR-8	State Primitives and Sessions	7 data primitives, dotted paths, sessions, disposable clones	—
VDR-9	Orchestrated Inference	Inference loop, 4 modes, notebooks, exact confidence	—

ID	Title	Central Result	Tests
VDR-10	Operational Foundations	IOSE model, 15 engineering principles, 448 builtins	—
VDR-11	Implementation Blueprint	5-stage build plan, 65 modules, 533 IOSE declarations	—
VDR-12	Grammar-Directed Compaction	Compaction format, grammars as KB fields, per-token provenance	179
VDR-13	Physical Computation	14 physical domains, 12 float failure points, complex pairs	—
VDR-14	Complete System Spec	Consolidation of 13 papers into single specification	—
VDR-15	Prompt Optimization	Token economics, 85-97% elimination, scaling analysis	—
VDR-16	Structural Safety	Visibility checks, grants, default denial, zero token cost	—
VDR-17	Alignment	Helpful/harmless/honest-through structure, not behavior	—
VDR-18	GPU Performance	5 concurrent streams, grammar-constrained decode, primitive scheduling	—
VDR-19	Self-Extension	Accumulation curve, $L1 \rightarrow L2 \rightarrow L3$, seed layers, bootstrap	—
VDR-20	Operational Deployment	4 runner types, coverage loop, directory interface, scaling	—

ID	Title	Central Result	Tests
VDR-21	FPGA	10-core Q335 processor, 53-instruction ISA, 6 microprograms	—
VDR-22	Dedicated Silicon	5,120 QIU ASIC, 5.1T mul/sec, zero-gate SHR335	—
VDR-23	Functional Remainder HW	FRU per QIU, adaptive depth, continuous resolution, 10M+ sessions	—
VDR-24	LLM Software	Sessions as applications, snapshot as binary, clone as process	—
VDR-25	LLM Server Software	Protocol grammars, clone-per-request, exact accounting	—
VDR-26	VDR and Diffusion	Zero-drift denoising, exact forward-reverse roundtrip	—
VDR-29	Zig SIMD and GPU Perf	Fixed-basis integer on production hardware, H100 INT8 projections	—
VDR-30	Economics of Scale	Dollar and watt costs, datacenter fleet projections	—
VDR-31	VDR Toy LLM	Python exact transformer, 181 params, worst softmax residual 9.07×10^{-13}	—
VDR-32	VDR-Zig Q16 LLM	688 ns/forward, 1.42M tok/sec, instruction-level INT8 equivalence	—
VDR-33	Compound Architecture	Five-axis multiplication, 71 $\times$ single session, $\sim 8,000 \times$ mature	—

ID	Title	Central Result	Tests
VDR-34	This paper	Mechanical accounting of all compound benefits	—

Cumulative across validated papers: 884 tests, zero VDR computation errors, 37 domains.

[[HOWL-VDR-34-2026](#)] Why Exact Integer Arithmetic Changes Everything About LLM Systems. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-34-2026>

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-14-2026>

[[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-21-2026>

[[HOWL-VDR-22-2026](#)] VDR-LLM-Prolog on Dedicated Silicon. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-22-2026>

[[HOWL-VDR-23-2026](#)] VDR-LLM-Prolog: Functional Remainder Hardware. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026>

[[HOWL-VDR-24-2026](#)] LLM Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-24-2026>

[[HOWL-VDR-25-2026](#)] LLM Server Software. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-25-2026>

[[HOWL-VDR-26-2026](#)] VDR and Diffusion. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-26-2026>

[[HOWL-VDR-27-2026](#)] VDR Beyond Language Models. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-27-2026>

[[HOWL-VDR-28-2026](#)] Exact Rational Arithmetic for Sequential Computation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-28-2026>

[[HOWL-VDR-29-2026](#)] VDR in Zig SIMD and GPU Performance versus Floating Point. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-29-2026>

[[HOWL-VDR-30-2026](#)] Economics of Scale: Floating Point vs Exact Integer ML Models. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-30-2026>

[[HOWL-VDR-31-2026](#)] VDR Toy LLM. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-31-2026>

[[HOWL-VDR-32-2026](#)] VDR-Zig Q16 Integer LLM. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-VDR-32-2026>

[[HOWL-VDR-33-2026](#)] VDR-LLM-Prolog: The Compound Architecture Performance Gains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-33-2026>

[[HOWL-VDR-3-2026](#)] VDR Gym Extension. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-3-2026>

[[HOWL-VDR-5-2026](#)] Exact Arithmetic Meets Logical Provenance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-5-2026>

[[HOWL-VDR-10-2026](#)] Operational Foundations and Comprehensive Builtin Specification. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-10-2026>

[[HOWL-VDR-12-2026](#)] Grammar-Directed Compaction and Generation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-12-2026>

[[HOWL-VDR-16-2026](#)] VDR-LLM-Prolog: Safe by Contract. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-VDR-16-2026>

[[HOWL-VDR-8-2026](#)] Computational State Primitives, Universal Data Pathing, and Session Management. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-8-2026>

[[HOWL-VDR-15-2026](#)] VDR-LLM-Prolog: Prompt Optimization. Github:
<https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-15-2026>

[[HOWL-VDR-19-2026](#)] VDR-LLM-Prolog: Self-Extending Architecture. Github:
<https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-19-2026>